

前言：一个“Reactor”的小故事（可忽略）

在一个叫做**并发国度**的地方，开发者们每天都在为“高并发”而头疼。最早，他们只会写“阻塞式”的网络程序：对每个连接都创建一个线程来处理，结果线程爆炸式增长，占用大量内存，性能也很快遇到瓶颈。

后来，他们学会了 `select` 和 `poll`，通过**I/O多路复用**来同时处理多个连接。可是当连接数进一步增加，`select` 和 `poll` 的效率又跟不上了，操作复杂不说，频繁的轮询也浪费了大量CPU资源。

不久之后，他们迎来了新的福音——`epoll`。`epoll` 在处理大量连接时性能极佳，只需少量线程就能支持海量并发。然而，随着项目规模扩大，代码开始变得凌乱：哪里监听、哪里读写、哪里存放回调处理逻辑，全都绞在一个大循环里，开发者们慢慢地陷入了泥沼。

有一天，一个叫Reactor的设计模式出现在他们面前。Reactor声称：“我可以把**事件分发与业务处理**彻底分离，大家只需要在我这里注册回调，就能轻松管理所有socket事件！而且还特别容易扩展，多线程、多进程、主从Reactor架构，都可以慢慢加上去！”

于是，大家决定从零开始编写一个**简易版Reactor**库，好让他们的高并发服务器轻装上阵，提高代码可读性与可维护性，最终释放出无穷的性能潜力。

开发者们知道，虽然学会Reactor不能一蹴而就，但只要坚定信念，一步步实现并测试，Reactor之路定会越走越远。

什么是Reactor模型

随着互联网的发展，服务器端需要同时处理上万、甚至更多连接。例如大型即时通信、游戏后端、高并发 Web Server。传统的“一个连接一个线程”在海量连接时会导致线程资源耗尽、频繁上下文切换，系统性能大打折扣。为了解决此问题，操作系统引入**非阻塞I/O**和**I/O多路复用**手段，在 Linux 上，`epoll` 就是其中最具代表性的高性能多路复用 API 之一。它可以在**单线程或少量线程**的情况下，高效处理成千上万的socket事件。

然而，如何有效管理这些事件，避免将业务逻辑和网络逻辑耦合在一起？**Reactor模型**给出了非常优雅的答案。它将**事件管理与实际处理**区分开，并通过**回调或虚函数**的方式进行业务逻辑的“注入”，从而带来结构清晰、模块化设计、可扩展性强等优势。

Reactor是一种**事件驱动**（event-driven）的并发模型/设计模式，最先由 Douglas C. Schmidt 在他的论文中提出。在这种模型中，应用程序不再为每个连接创建一个阻塞式线程，而是将所有连接事件统一交给**“事件分离器（event demultiplexer）”去监控。当某个连接产生事件（可读、可写、异常等），这个事件分离器会将事件通知给Reactor**，后者再调用相应的**事件处理器（Handler）**来执行业务逻辑。

这种思路很像**GUI编程**：大家应该学习了我们的windows编程的课程了，我们在图形界面中，会注册一个按钮点击事件的回调，当按钮被点击时就自动调用回调函数。Reactor同理，只不过在网络编程里，“按钮点击”换成了“socket可读或可写”等事件。

Reactor 与 Proactor

在谈到 Reactor 时，总会提到它的“兄弟”——Proactor。两者都是事件驱动模型，但**最大区别**在于：

1. **Reactor**：内核只负责通知“可读”或“可写”，真正的读写操作由用户态来完成；！
2. **Proactor**：真正的读写由内核或系统级别的异步I/O机制完成，用户态只在“读写完成”时获得通知，并做后续处理。在 Linux 环境下，一般使用“同步非阻塞 I/O + epoll + Reactor”这一套组合来实现高并发服务器。**Proactor**多见于 Windows 上的 IOCP 等。！32 在我们后续的项目课程中，远控会用到Proactor，结合IOCP！

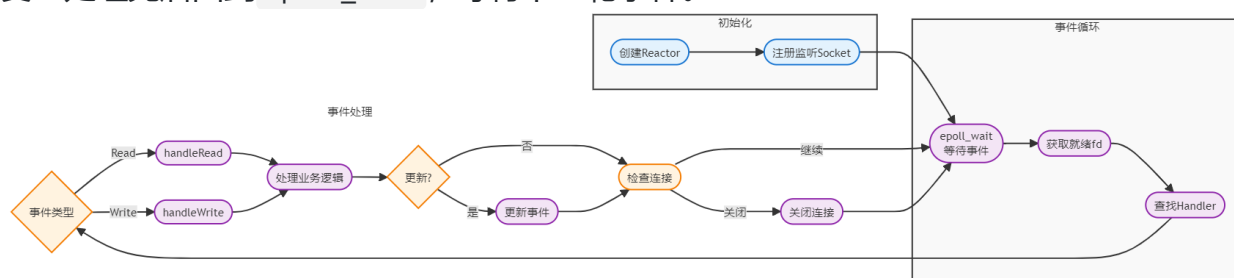
Reactor模型的重要组成

Reactor模型通常包含如下几大角色：Reactor（反应器）！

- 管理所有感兴趣的事件（通常是网络socket上的可读、可写、错误、关闭等）。
- 与底层的多路复用接口打交道（如 `epoll_wait`、`select`、`poll` 等）。
- 当事件准备好后，它把事件“分发”给相应的处理器（Handler）执行处理逻辑。
- **Event Demultiplexer（事件分离器）！**
 - 这是操作系统层面的接口或对此的封装，用来“收集”所有socket事件并统一返回就绪事件给Reactor。
 - 在Linux上，最典型的实现就是 `epoll`。你可以视其为Reactor和操作系统之间的中间件。
- **Handler（事件处理器）！**
 - 一个Handler对应特定的事件或socket连接。
 - 它会实现若干回调函数，如“当可读时如何处理数据”“当可写时如何发送数据”“当出错或关闭时该做哪些清理”等。
 - Handler的实现跟业务逻辑紧密相关，比如EchoHandler、HttpHandler、或者游戏服中的RoomHandler等等。
- **Acceptor（可选）！**
 - 对服务器端来说，还需要专门“接受新连接”的角色。
 - 当监听socket上有新连接到来时，就会触发Acceptor的回调，进行 `accept()` 操作，并把新的连接socket交给其他Handler进行后续管理。

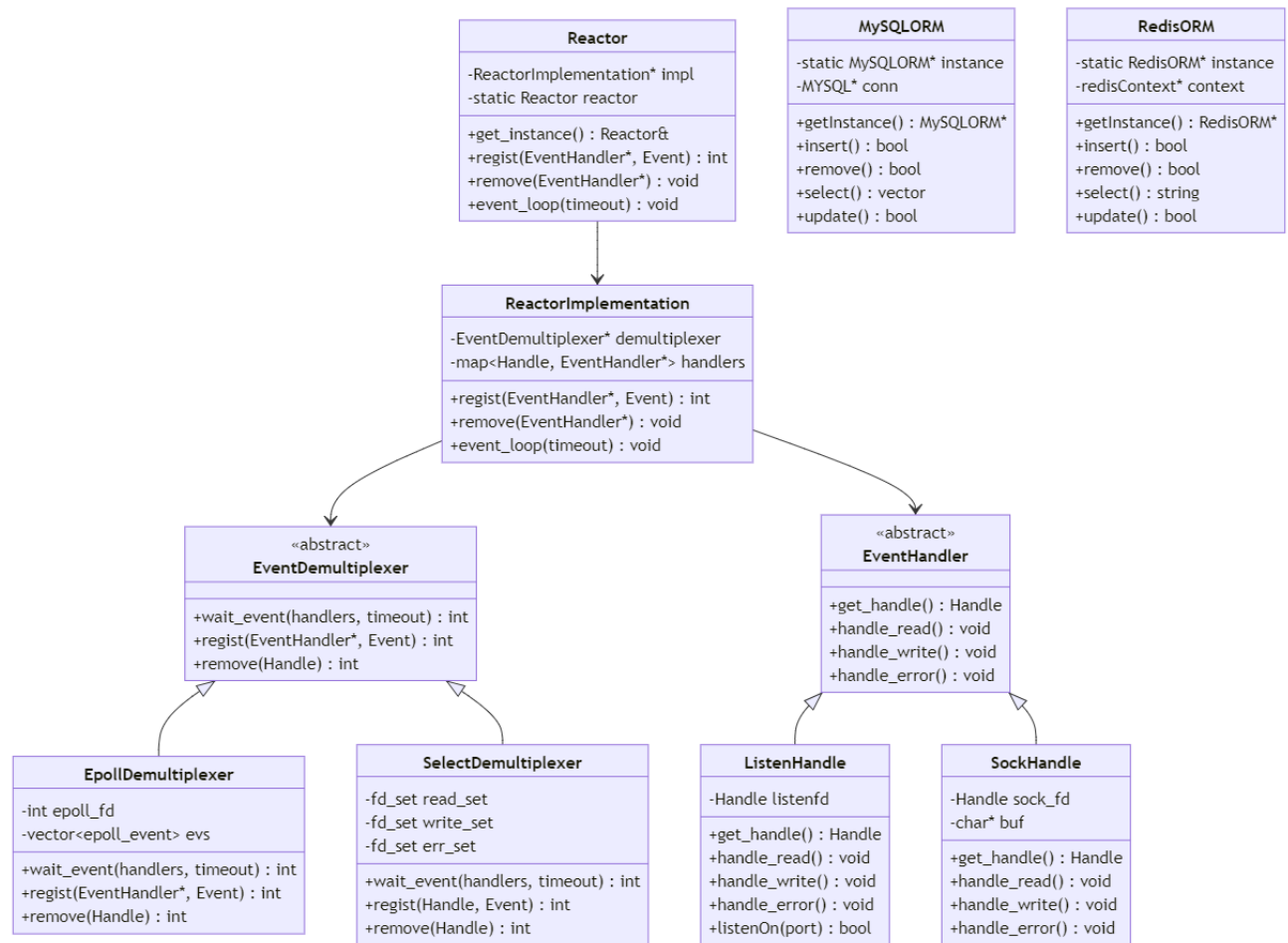
一个最小化的单Reactor场景大致如下：

1. **初始化**：创建Reactor对象并把监听socket注册上；
2. **事件循环**：进入死循环，Reactor在 `epoll_wait` 阻塞；
3. **事件到来**：某些fd就绪时，Reactor获取它们；
4. **调用回调**：根据fd找到其对应Handler，调用 `handleRead()` 或 `handleWrite()` 等函数；
5. **处理完成**：Handler可能会更新所关心的事件（例如，从只读变为可写或相反），或者移除/关闭连接；
6. **重复**：处理完后回到 `epoll_wait`，等待下一轮事件。

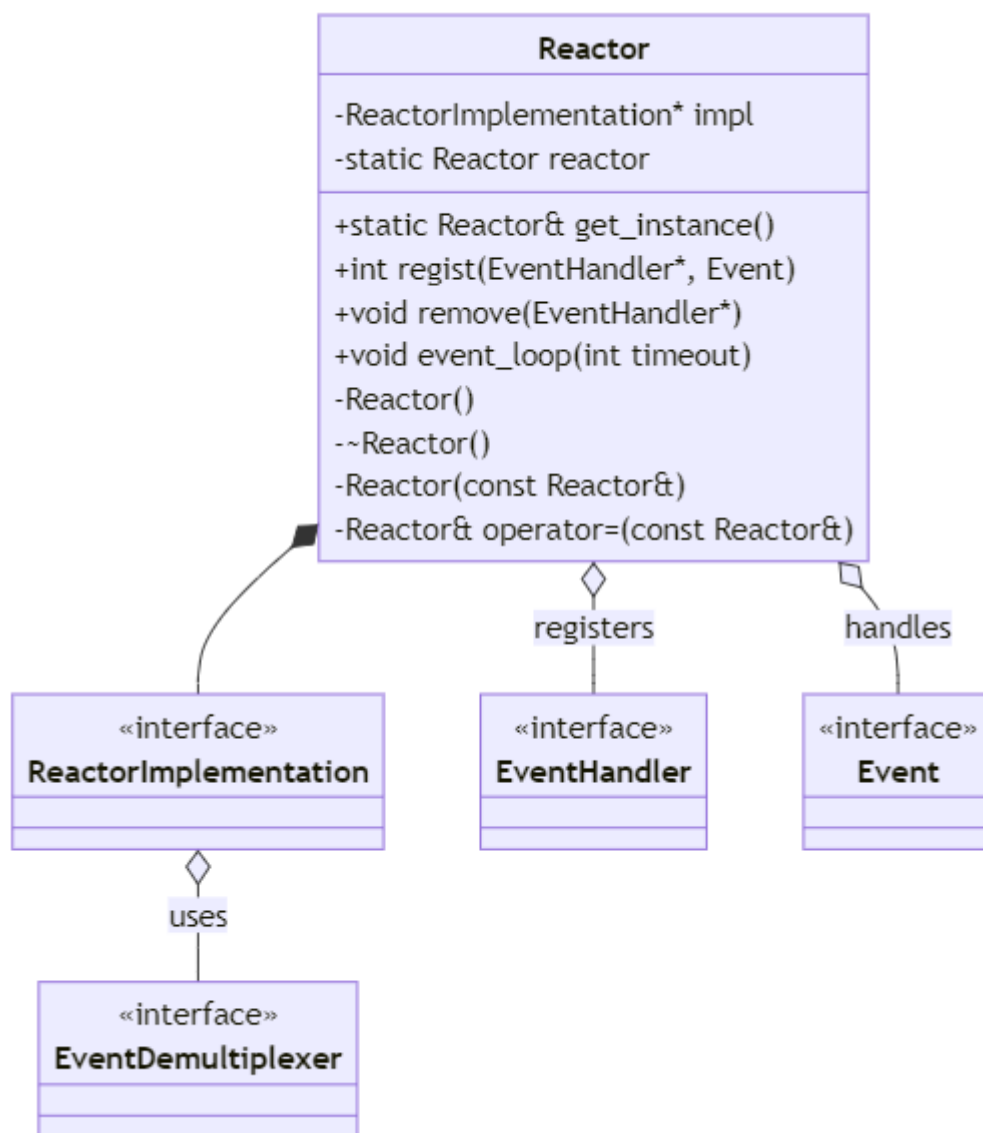


在具体实现一个Reactor之前，必须搞清楚它的几个重要组成及作用，以便在设计阶段完成良好架构划分。这里我们不写具体代码，而是将其看作几个抽象模块，给出设计要点与示意图！

#系统架构与设计 首先我们先展示整体的类图：

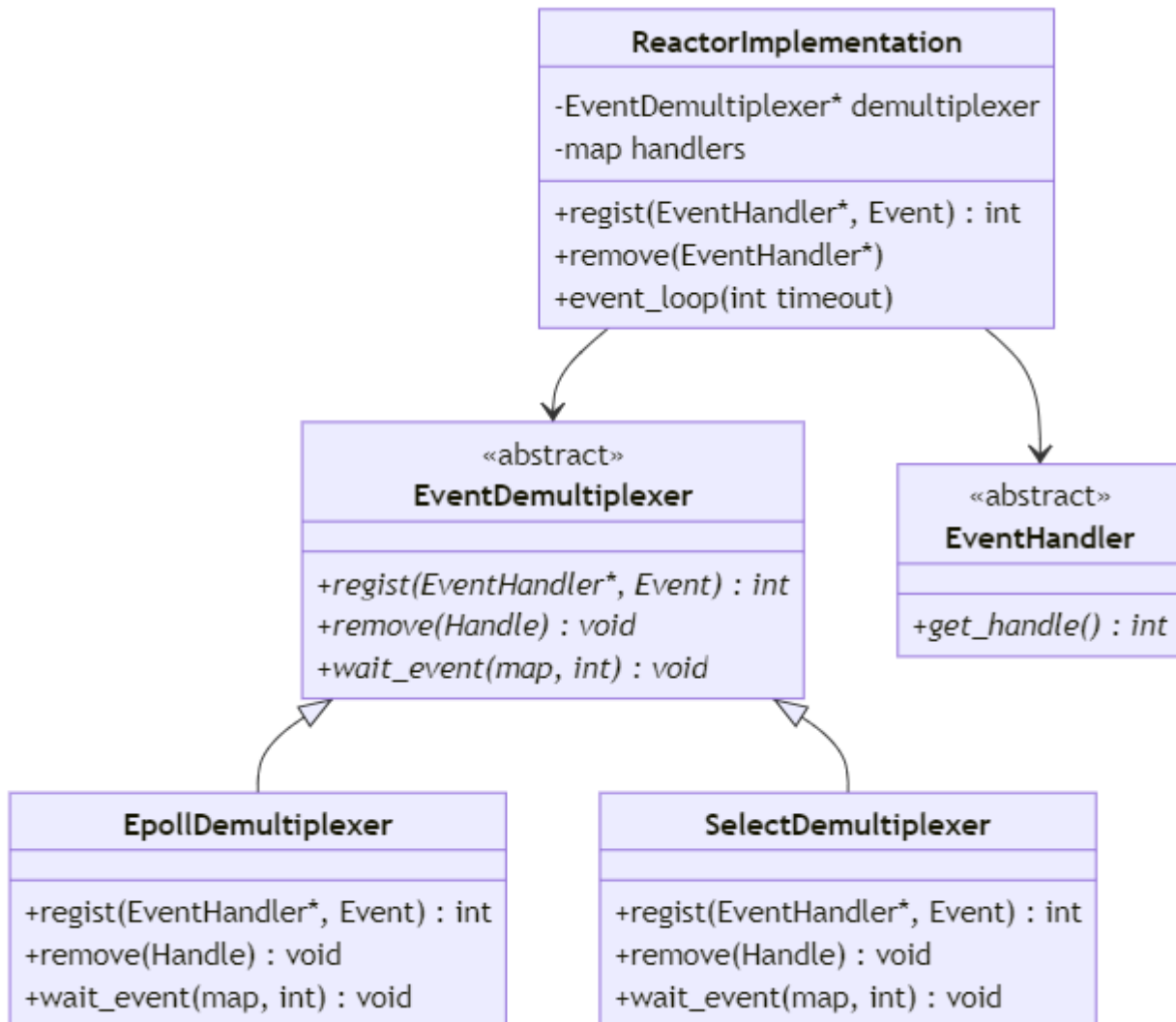


Reactor类（接口类）



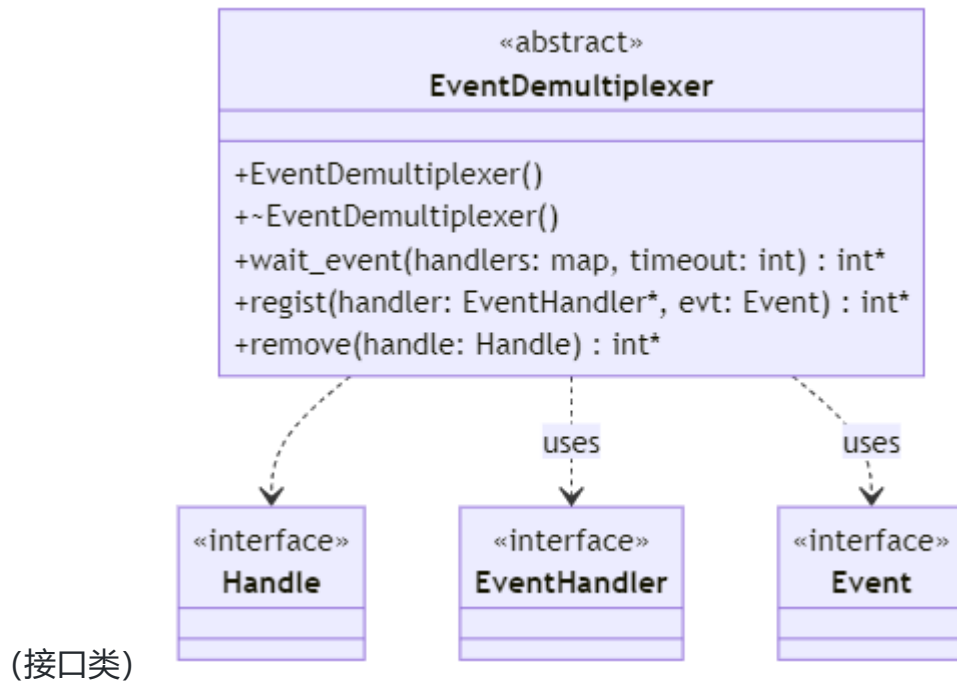
- Reactor类
- 使用单例模式实现，通过 `get_instance()` 获取唯一实例
- 通过 `impl` 指针桥接到具体实现，实现了接口和实现的分离
- 提供了三个主要操作：注册事件(`regist`)、移除事件(`remove`)和事件循环(`event_loop`)
- 相关接口：
 - **EventHandler**：事件处理器接口，定义了具体的事件处理方法
 - **Event**：事件接口，定义了事件的属性和行为
 - **EventDemultiplexer**：事件分发器接口，负责事件的检测和分发
 - **ReactorImplementation**：实际的反应器实现接口

##ReactorImplementation类



- `ReactorImplementation` 是反应器的核心实现类，负责事件的注册、移除和事件循环
- `EventDemultiplexer` 是事件多路分发器的抽象基类
- `EpollDemultiplexer` 和 `SelectDemultiplexer` 是具体的事件多路分发器实现
- `EventHandler` 是事件处理器的抽象基类，用于处理具体的事件 **主要组件功能：**
- `ReactorImplementation` 类：
 - 维护一个事件处理器的映射表（handlers）
 - 提供事件注册（regist）和移除（remove）接口
 - 通过 event_loop 方法进行事件循环处理

- 使用条件编译 (#ifdef) 来选择使用 epoll 或 select 作为底层实现 ##EventDemultiplexer类



1. 整体设计目的： EventDemultiplexer（事件多路分解器）是反应器模式（Reactor Pattern）中的一个核心组件，用于处理和分发各种I/O事件。它的主要职责是监控多个事件源，当事件发生时，将其分发给相应的事件处理器。

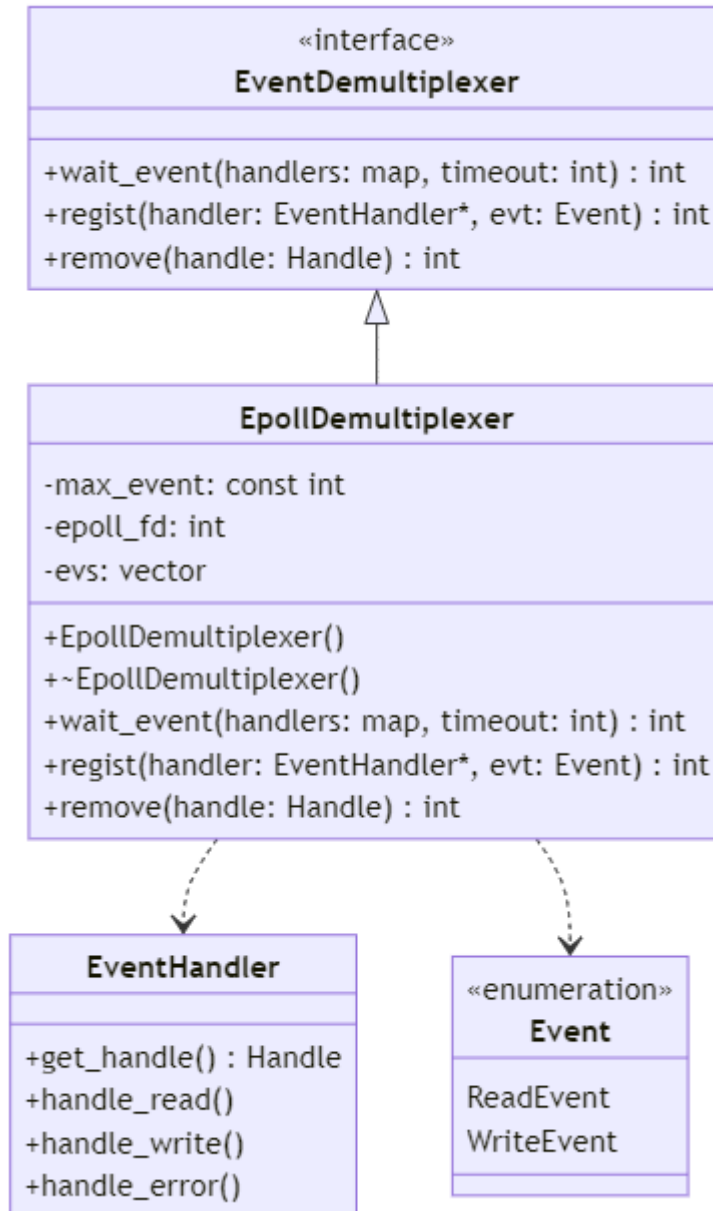
2. 类的主要组件：

- EventDemultiplexer 是一个抽象基类，定义了事件多路分解器的基本接口
- 包含三个纯虚函数，要求子类必须实现这些功能
- 使用了智能指针和STL容器来管理事件处理器

1. 核心方法说明：

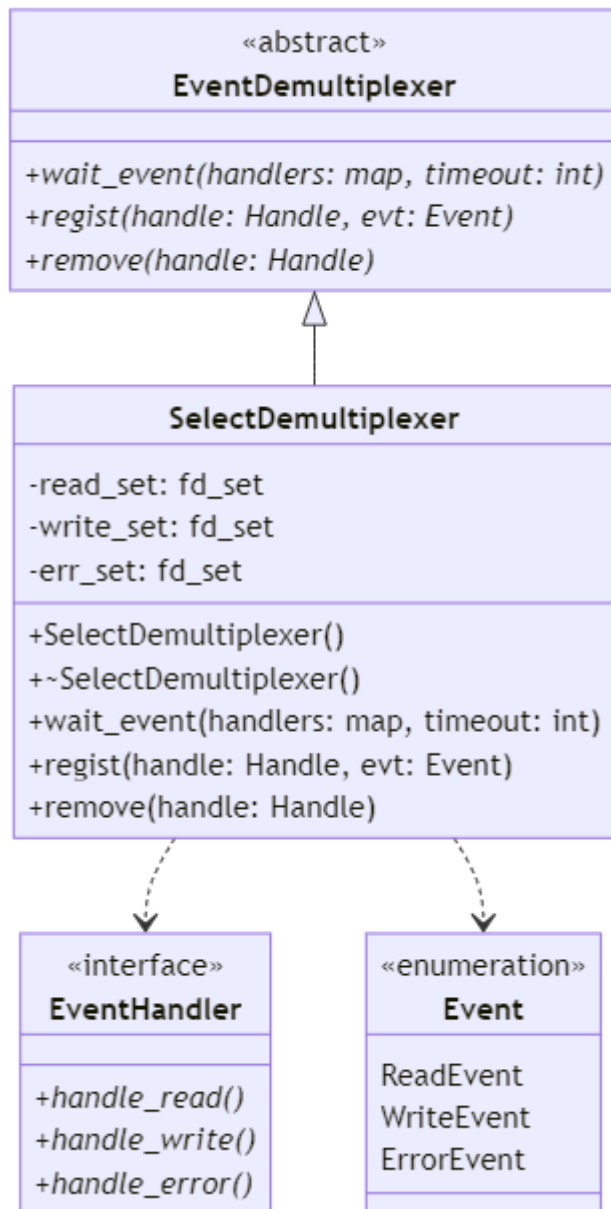
- `wait_event`：等待事件发生，参数包括：
 - `handlers`: 存储句柄和对应处理器的映射表
 - `timeout`: 超时时间，默认为0
 - 返回值表示处理的事件数量
- `regist`：注册新的事件处理器，参数包括：
 - `handle`: 事件处理器指针
 - `evt`: 要监听的事件类型
 - 返回注册是否成功
- `remove`：移除特定的事件处理器
 - `handle`: 要移除的处理器句柄

- 返回移除操作是否成功 ##EpollDemultiplexer类



- 继承自 EventDemultiplexer
- 使用 epoll 机制实现具体的事件多路分发
- 主要成员：
 - `epoll_fd`: epoll 实例的文件描述符
 - `evs`: 用于存储 epoll 事件的vector容器

- max_event: 最大事件数量 (1024) ##select_demultiplexer类



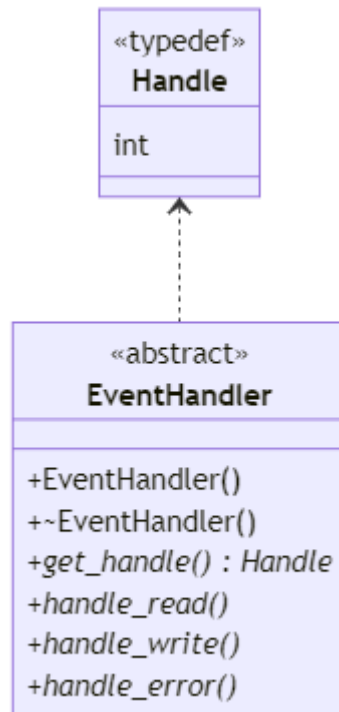
`SelectDemultiplexer` 类:

- 继承自抽象类 `EventDemultiplexer`
- 使用 `select` 机制监控文件描述符的读/写/错误事件
- 维护三个 `fd_set` 集合: `read_set`、`write_set` 和 `err_set`
- 实现了事件注册 (`regist`)、移除 (`remove`) 和等待 (`wait_event`) 功能

1. 主要方法说明: `regist` 方法:

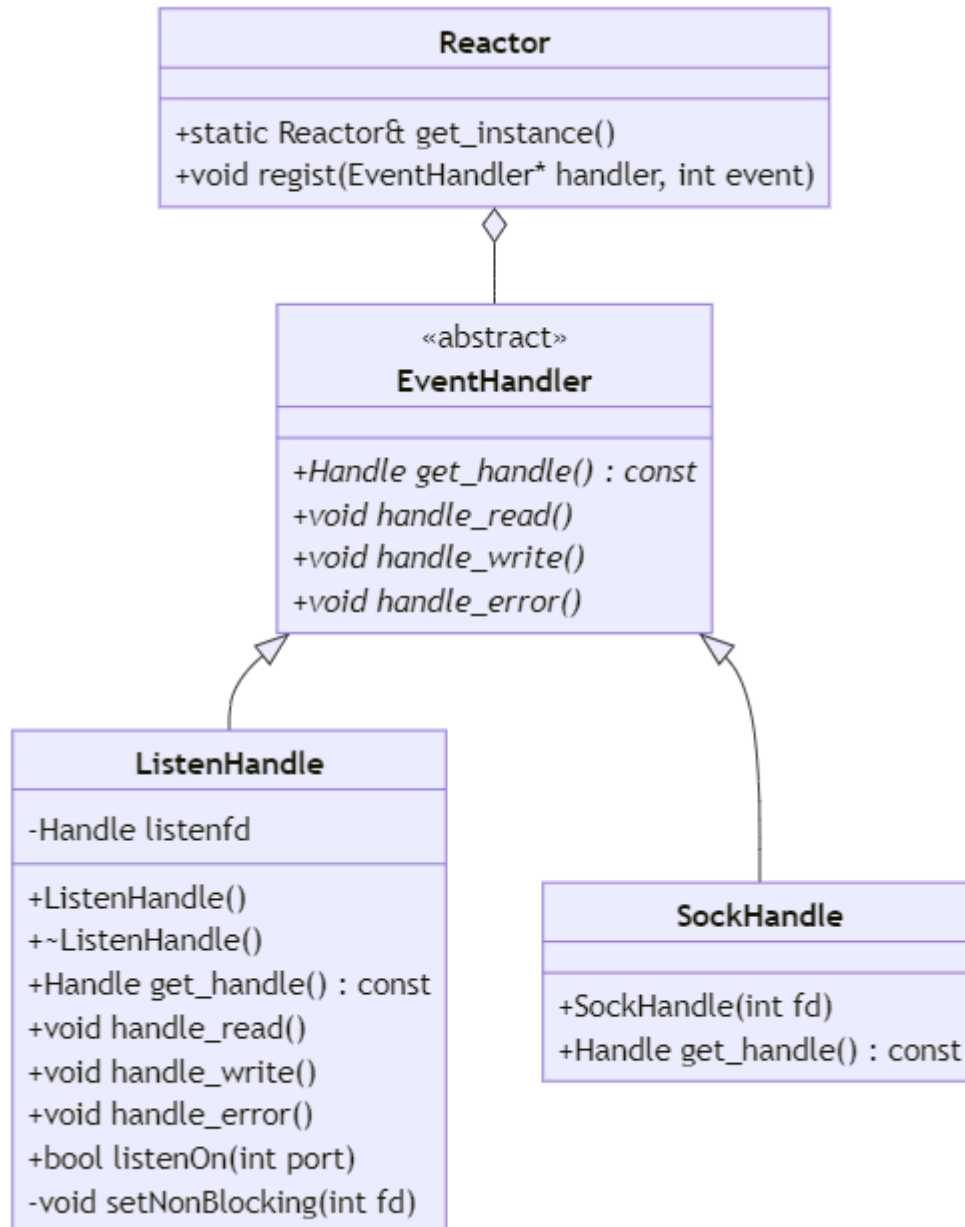
- 将文件描述符注册到相应的事件集合中
- 可以同时注册读事件和写事件
- 错误事件默认都会被监控 `remove` 方法:
- 从所有事件集合中移除指定的文件描述符 `wait_event` 方法:
- 核心方法, 用于等待和分发事件

- 使用 select 系统调用监控所有注册的文件描述符
- 当事件发生时，调用对应的处理器（EventHandler）进行处理

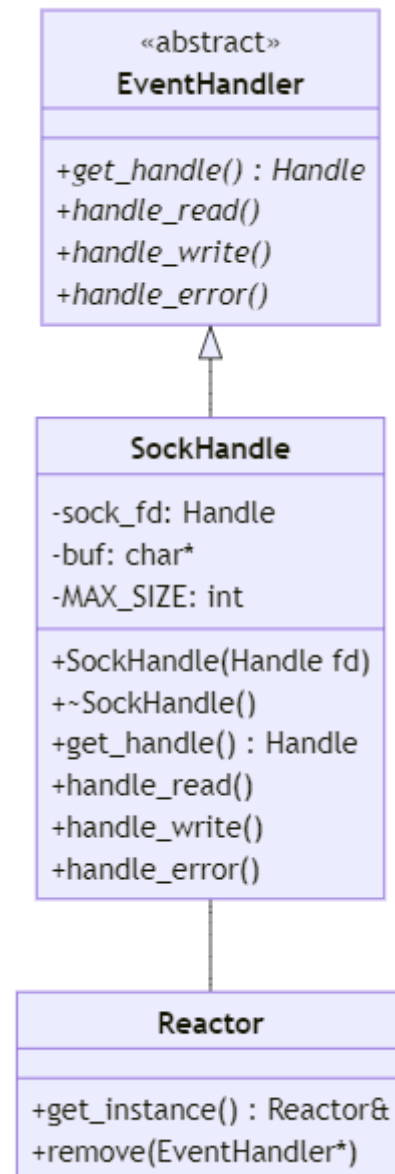


- 支持超时机制 `##event_handle`类（接口类）
 - 首先定义了类型别名 `typedef int Handle`，将 `Handle` 定义为整型，通常用于表示文件描述符或socket等资源句柄
 - `EventHandler` 是一个抽象基类，包含了一组纯虚函数（接口）
- `EventHandler` 类的主要组成：
 - 构造函数：默认构造函数
 - 虚析构函数：确保正确释放子类资源
 - 四个纯虚函数：
 - `get_handle()`：获取处理器关联的句柄
 - `handle_read()`：处理读事件
 - `handle_write()`：处理写事件

- `handle_error()` : 处理错误事件 `##listen_handle`类



- 继承自 `EventHandler`
- 负责处理服务器端的监听socket
- 实现了新连接的接受和处理



- 提供了端口监听和非阻塞设置功能 ##sock_handle类
- - 继承自EventHandler
 - 负责具体的socket读写操作
 - 维护一个固定大小(1024字节)的缓冲区
 - 实现了回显(echo)功能

1. 主要功能特点:

- 读取处理(handle_read):
 - 从socket读取数据到缓冲区
 - 将读取的数据直接写回socket(echo功能)

二、项目功能要求

1. 封装 epoll (必做)

- 写一个类，如 `EpollDemultiplexer`，提供 `addEvent(fd, events)`，`modifyEvent(fd, events)`，`removeEvent(fd)`，`waitEvents(...)` 等接口。
- 使用少量或单次 `new`，不可滥用内存分配。可参考“缓冲区”理念，反复使用同一数组或容器来接收就绪事件。
- 需要有完整错误提示或返回值检查。

2. Reactor 类 (必做)

- 提供 `registerHandler(fd, handler, events)`，`removeHandler(fd)`，`modifyHandler(fd, events)`，`eventLoop()` 等；
- 内部维护 `fd->Handler` 映射，`eventLoop()` 中调用 `epoll_wait` 等；
- 对就绪事件，能正确地调用相应Handler的 `handleRead()`、`handleWrite()`、`handleError()`、`handleClose()`。

3. Handler 抽象基类 + Acceptor + 连接Handler (必做)

- `Handler` 要有最基本的四个函数，子类必须实现；
- `Acceptor`：只负责监听套接字的 `accept` 操作，把新连接 `fd` 交给连接Handler管理；
- `ConnectionHandler`：处理客户端读写，例如实现一个 `Echo` 或更简单的收发测试即可。

4. 多线程或主从Reactor (选做，加分)

- 在 `handleRead()` 中只做数据读取，然后把数据交给线程池去做耗时处理，处理完再写回；
- 或者实现一个“主Reactor” + “从Reactor”结构，主Reactor负责 `listenfd` 事件，从Reactor管理普通连接。
- 需要保证线程安全性。

提交材料

1. **源代码**：包括头文件与实现文件，建议使用CMake或Makefile；

2. **运行演示**：至少说明如何在Linux环境下编译 (`g++ -o server *.cpp -lpthread ...`) 并执行 (`./server`)

评分标准

- **功能完整度 (50%)**：Reactor核心功能+Acceptor+EchoHandler是否正确实现；
- **代码质量 (20%)**：是否结构清晰、有合理的注释、无明显内存泄漏；
- **加分项 (30%)**：使用多线程，或实现主从Reactor结构，或有定时器机制等。